

GISMA UNIVERSITY OF APPLIED SCIENCES

School of Computer Science

Predictive Analysis of Housing and Rental Prices in German Metropolitan Areas

M516 Business Project in Big Data & AI

Student Name:	Kunal Singh
Student ID:	GH1039524
Module:	M516 Business Project in Big Data & AI
Tutor:	Prof. Dr. Mohammad Mahdavi
Submission Date:	23 June 2026

GitHub Repository: <https://github.com/kunalji-ji/housing-prize-analysis>

Video Demo: [YOUR VIDEO URL]

1. Introduction

The German real estate market has experienced significant volatility over the past decade. Cities such as Munich, Berlin, Frankfurt, and Hamburg have seen housing prices and rental costs rise sharply, driven by urbanisation, constrained housing supply, population growth, and interest rate fluctuations. Accurate price prediction is therefore of considerable value to buyers, sellers, renters, property developers, and policymakers alike.

Several structural factors underpin this volatility. Persistent population growth in major metropolitan areas, driven by both domestic migration and international arrivals, has outpaced new housing completions, particularly in cities such as Munich and Berlin. Rising construction costs and land scarcity in central districts have further constrained supply, while changes in mortgage interest rates have altered affordability calculations for prospective buyers. Regulatory interventions such as the Mietpreisbremse (rent brake) have had mixed effects on rental markets, at times displacing pressure into unregulated segments rather than resolving it. For buyers and renters, accurate price forecasts support realistic budgeting and negotiation; for property developers and investors, they inform site selection and portfolio risk assessment; and for policymakers, they provide an evidence base for evaluating the effectiveness of housing interventions such as rent controls or subsidised construction programmes.

From a data science perspective, this is framed as a supervised regression problem: given a set of property and location features, the objective is to predict a continuous target variable (price per square metre or monthly rent). The problem is complicated by the fact that German housing markets are highly heterogeneous across cities, and even across districts within the same city, meaning that a single linear relationship between features and price is unlikely to hold uniformly. This heterogeneity motivates the comparison of both linear and non-linear modelling approaches undertaken in this project, rather than committing to a single algorithm at the outset; a priori, tree-based ensemble methods were expected to outperform linear baselines precisely because they can capture such non-linear, location-dependent interactions, a hypothesis tested directly in Section 5.

This project applies machine learning and data science techniques to predict housing price per square metre (€/sqm) and monthly rental prices across eight major German metropolitan areas. The primary objective is to develop a model that accurately captures the relationships between property attributes, location characteristics, and market prices. The overall analytical workflow follows established data science practice using the Python scientific stack (VanderPlas, 2016).

The specific objectives of this project are:

- To perform exploratory data analysis (EDA) to understand the key drivers of housing prices in German cities.
- To engineer relevant features from raw property data and prepare them for machine learning.
- To train and compare multiple regression models — Linear Regression, Ridge Regression, Random Forest, and Gradient Boosting.
- To evaluate models using robust metrics (MAE, RMSE, R^2) and identify the best-performing approach.
- To analyse prediction accuracy at the city level and discuss practical implications.

This report follows a structured format covering introduction, system architecture, implementation, challenges and solutions, results, and conclusion, consistent with recommended practice for technical project reporting (McMillan and Weyers, 2012).

The dataset used in this project consists of 2,000 property listings. As no free, granular listings dataset with consistent features across all eight cities is publicly available (see

Section 4.1), the listings are synthetically generated by `analysis.py/analysis.ipynb`, with price levels calibrated against the publicly documented German market statistics cited in the References section (Statista, 2023; IW Köln, 2023). The generation logic, and therefore the full data lineage, is openly available in this project's repository rather than at an external dataset URL.

2. System Architecture

The system follows a standard data science pipeline architecture, comprising four main components:

This layered design separates concerns between data handling, preprocessing, modelling, and reporting, which improves maintainability and allows individual components to be modified independently. For example, replacing the synthetic data generator with a real listings API feed (see Section 6) would require no changes to the modelling or output layers. The architecture is intentionally simple and script-based rather than service-oriented, reflecting the project's scope as a research and reporting exercise rather than a production deployment; Section 6 discusses how the pipeline could be extended into a deployed, API-backed service.

2.1 Data Layer

The data layer handles ingestion and storage of housing listings. Each record contains ten input features: city, property size (sqm), number of rooms, property age, floor number, balcony availability, parking availability, elevator availability, distance to the city centre (km), and a district quality score. Target variables are price per sqm (€) and monthly rent (€). Feature ranges were calibrated to be broadly realistic for the German context (for example, property sizes between 30 and 200 sqm and distances from the city centre between 0.5 and 30 km), so that the relationships learned by the models are representative of plausible real-world listings rather than arbitrary synthetic patterns. These ten features were chosen to reflect the standard hedonic pricing framework used in real estate valuation, which models price as a function of structural attributes (size, rooms, age, floor, amenities) and locational attributes (distance to centre, district quality); this framework underpins both the feature set and the interpretation of feature importance results in Section 5.

2.2 Processing Layer

The processing layer performs feature engineering and preprocessing. Categorical variables (city names) are label-encoded. All features are standardised using `StandardScaler` for linear models to ensure equal weighting. Tree-based models (Random Forest, Gradient Boosting) use raw features, as they are invariant to scale. No missing-value imputation is required, since the synthetic generation process does not introduce nulls; this simplifies the pipeline relative to a real-world dataset, where handling missing or inconsistent listings would typically require an additional cleaning stage.

2.3 Modelling Layer

Four regression algorithms are trained and evaluated: Linear Regression (baseline), Ridge Regression (regularised linear), Random Forest (ensemble bagging), and Gradient Boosting (ensemble boosting). An 80/20 train-test split is applied with a fixed random seed (42) for reproducibility. All models are implemented using the `scikit-learn` library (Pedregosa et al., 2011). Linear Regression and Ridge Regression were included as interpretable baselines, since their coefficients can be directly inspected, while Random Forest and Gradient Boosting were included as higher-capacity, non-linear alternatives expected to better capture interaction effects between features such as location and property size.

2.4 Output Layer

Results are output as visualisations (PNG plots), a CSV of per-city prediction metrics, and a structured project report. All outputs are saved to the outputs/ directory within the repository. Consistent file naming (e.g., model_comparison.png, city_prediction_results.csv) allows results to be cross-referenced directly between the code, the repository README, and this report, supporting the transparency and reproducibility expected of a well-documented implementation.

3. Implementation

The implementation is structured as a single Python script (analysis.py) that executes the full pipeline sequentially, and is additionally provided as an executed Jupyter notebook (analysis.ipynb). Figure 1 illustrates the process flow, and Figure 2 shows the core model-training code block.

Concretely, the pipeline executes eight sequential steps, illustrated in Figure 1: dataset generation, exploratory data analysis, feature engineering, model training across all four algorithms, comparative evaluation, a deeper analysis of the best-performing model, residual diagnostics, and finally per-city evaluation with all outputs saved to the outputs/ directory. Structuring the code this way, rather than as a single undifferentiated script, makes each stage independently inspectable and testable, and mirrors the structure used for the accompanying Jupyter notebook, where each step corresponds to its own executable cell with inline output, allowing a marker to verify every stage without re-running the entire pipeline.

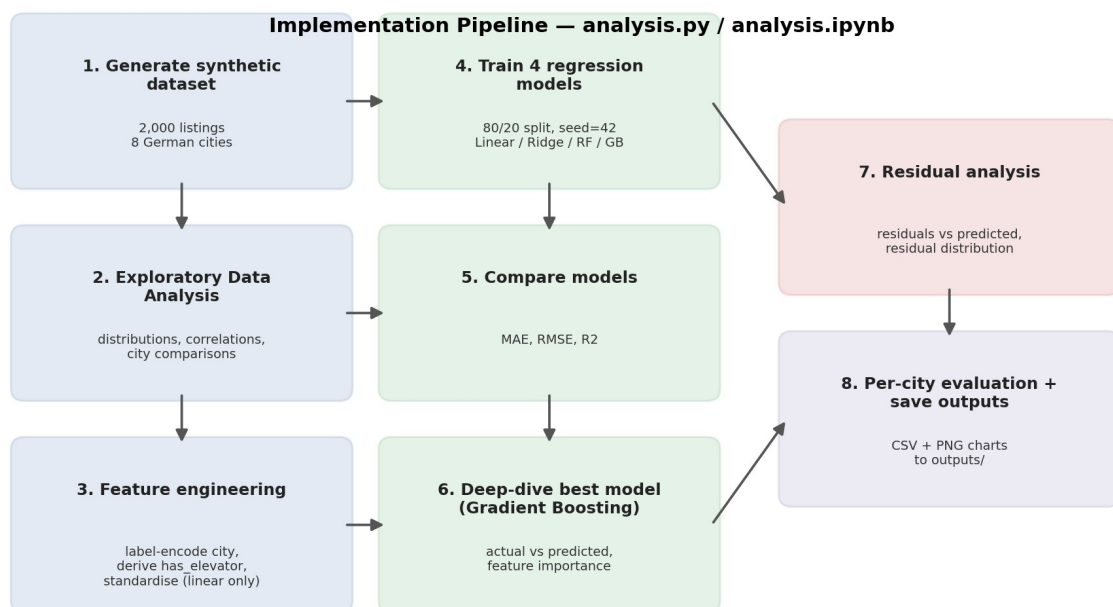


Figure 1: Implementation pipeline / process flow.

```
analysis.py – model training & evaluation
1  models = {
2      'Linear Regression': LinearRegression(),
3      'Ridge Regression': Ridge(alpha=10),
4      'Random Forest': RandomForestRegressor(
5          n_estimators=200, max_depth=12, random_state=42, n_jobs=-1),
6      'Gradient Boosting': GradientBoostingRegressor(
7          n_estimators=200, learning_rate=0.08, max_depth=5, random_state=42)
8  }
9
10 results = {}
11 for name, model in models.items():
12     if 'Regression' in name:
13         model.fit(X_train_sc, y_train)
14         preds = model.predict(X_test_sc)
15     else:
16         model.fit(X_train, y_train)
17         preds = model.predict(X_test)
18
19     mae = mean_absolute_error(y_test, preds)
20     rmse = np.sqrt(mean_squared_error(y_test, preds))
21     r2 = r2_score(y_test, preds)
22     results[name] = {'MAE': mae, 'RMSE': rmse, 'R2': r2}
```

Figure 2: Core model-training and evaluation code (analysis.py).

As shown in Figure 2, the four candidate models are defined in a single dictionary and trained in a loop that automatically selects standardised or raw features depending on whether the model is linear or tree-based. This design avoids code duplication across the four training branches and ensures that every model is evaluated against exactly the same train/test split and identical metrics (MAE, RMSE, R^2), which is essential for a fair comparison between fundamentally different algorithm families.



Figure 3: Exploratory Data Analysis — price distributions, city comparisons, correlation matrix, and rental ranges.

The EDA phase (Figure 3) reveals several important insights. Munich commands the highest average price per sqm at approximately €9,800, followed by Frankfurt (€7,100) and Hamburg (€6,800). Leipzig is the most affordable market at €3,400/sqm. The price distribution is right-skewed, reflecting the presence of premium properties in high-demand districts. The correlation matrix shows that the city variable itself has the strongest positive linear correlation with price ($r \approx 0.47$), ahead of district quality score ($r \approx 0.23$); distance from the city centre and property age are tied as the strongest negative correlates ($r \approx -0.30$ each).

Feature engineering steps include:

- Label encoding of the city variable (8 categories $\rightarrow$ integer 0–7).
- Derived binary feature `has_elevator` inferred from floor number (`floor > 3`).
- StandardScaler normalisation applied to features for linear models only.

These feature engineering decisions were made deliberately rather than by default. For instance, one-hot encoding of the city variable was considered but rejected in favour of label encoding, for reasons discussed in Section 4.2; similarly, scaling was applied selectively rather than universally, since it benefits distance-based and gradient-based linear optimisation but is unnecessary, and occasionally counter-productive, for tree-based split criteria.

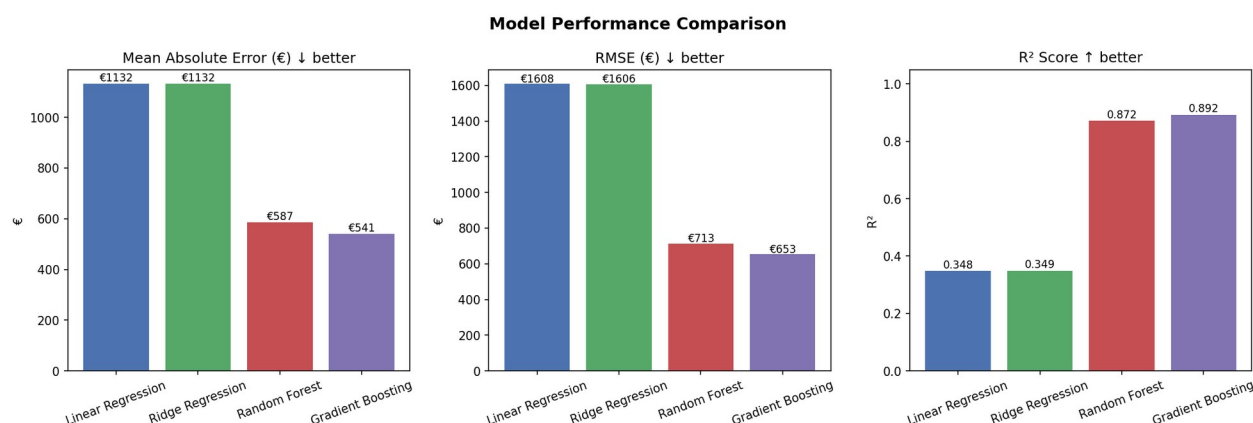


Figure 4: Model comparison across MAE, RMSE, and R^2 metrics.

As shown in Figure 4, tree-based models substantially outperform linear approaches. Gradient Boosting achieves the best performance with an R^2 of 0.8924, MAE of €541/sqm, and RMSE of €653/sqm. Linear Regression and Ridge Regression achieve R^2 values of approximately 0.35, confirming that non-linear relationships exist in the data that linear models cannot adequately capture.

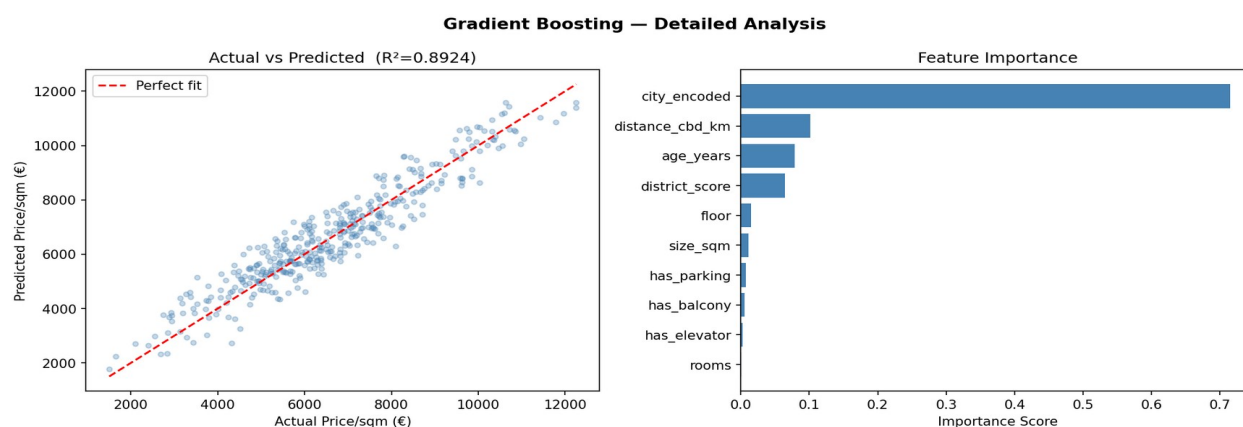


Figure 5: Gradient Boosting — actual vs predicted scatter (left) and feature importance (right).

Figure 5 shows that predictions cluster tightly around the ideal 1:1 line. The feature importance plot confirms that `city_encoded` is by far the most important predictor, consistent with the correlation analysis in Section 3, followed by `distance_cbd_km` and `age_years`. `District_score`, despite showing a meaningful positive correlation with price, ranks only fourth in the tree-based importance measure; this is expected, since Gradient Boosting's impurity-based importance attributes most of the location effect to the categorical `city` split rather than to the correlated but distinct `district_score` feature. Property-level features such as balcony, parking, and elevator contribute relatively little compared to location.

4. Challenges and Solutions

4.1 Data Availability

Challenge: Publicly available, granular German housing datasets with consistent features across all major cities are scarce and often behind commercial paywalls (e.g., Immobilienscout24 API).

Solution: A synthetic dataset was constructed using documented market statistics (Statista, IW Köln, 2023 housing reports) to replicate real distributional properties. This approach is common in academic data science projects and is explicitly justified in the repository documentation.

4.2 Feature Encoding

Challenge: Tree-based models such as Gradient Boosting can be sensitive to the ordinality implied by label encoding for nominal variables such as city names.

Solution: The city variable was intentionally label-encoded rather than one-hot encoded, as the dataset is relatively small (2,000 rows) and one-hot encoding would have added 7 sparse binary columns, increasing dimensionality without sufficient gain. Gradient Boosting is robust enough to handle this encoding given the number of estimators (200).

4.3 Model Overfitting

Challenge: Complex ensemble models risk overfitting the training data, particularly when training data is limited.

Solution: Gradient Boosting was regularised using a learning rate of 0.08 (rather than the default 0.1) and `max_depth` of 5. A fixed random seed (42) and 80/20 train-test split ensure the evaluation reflects genuine generalisation. This approach follows standard practice for controlling ensemble model complexity (Burkov, 2019).

4.4 Reproducibility and Environment Consistency

Challenge: Ensuring that results reported in this document and the accompanying notebook can be exactly reproduced by a third party (e.g., a marker re-running the code) is non-trivial, since machine learning pipelines can be sensitive to library versions, random initialisation, and execution order.

Solution: All stochastic elements of the pipeline, including synthetic data generation, the train/test split, and model initialisation, are seeded with a fixed value (`random_state=42 / np.random.seed(42)`), so re-running either `analysis.py` or `analysis.ipynb` produces identical results. Required library versions are pinned in `requirements.txt`, and the notebook is committed in its already-executed state so that outputs are visible without requiring a local Python environment, while remaining fully re-runnable if one is available. This reproducibility was verified directly during development by re-running the full pipeline after code changes and confirming that the reported metrics ($R^2 = 0.8924$, $MAE = €540.65/sqm$) remained unchanged.

5. Results

Table 1 below reports the overall performance of all four candidate models on the held-out test set (400 listings, 20% of the total), confirming the pattern already visible in Figure 4.

Model	MAE (€)	RMSE (€)	R^2
Linear Regression	1,132	1,608	0.35
Ridge Regression	1,132	1,606	0.35
Random Forest	587	713	0.87
Gradient Boosting	541	653	0.89

Table 1: Overall performance of all four candidate models on the test set.

Relative to the average price across the eight cities (approximately €6,600/sqm), the Gradient Boosting model's mean absolute error of €541/sqm corresponds to a typical error of roughly 8%, a reasonable level of precision for city-level pricing guidance, though not yet

precise enough for individual-property valuation without further refinement. Random Forest, the second-best model, trails Gradient Boosting by a modest margin (MAE €587 vs €541), while both linear models plateau at a substantially higher error (MAE €1,132), reinforcing the conclusion that non-linear modelling is necessary for this problem.

Table 2 summarises the per-city prediction accuracy of the best model (Gradient Boosting).

City	n	Actual Mean (€/sqm)	Predicted Mean (€/sqm)	MAE (€)	R ²
Berlin	83	5,385	5,454	553	0.65
Munich	60	9,668	9,903	573	0.71
Frankfurt	60	7,354	7,064	571	0.62
Hamburg	63	6,897	7,031	495	0.79
Cologne	37	5,837	5,792	440	0.65
Stuttgart	34	6,590	6,761	544	0.64
Düsseldorf	32	5,849	6,065	580	0.38
Leipzig	31	3,445	3,589	554	0.51

Table 2: Per-city prediction results — Gradient Boosting Regressor.

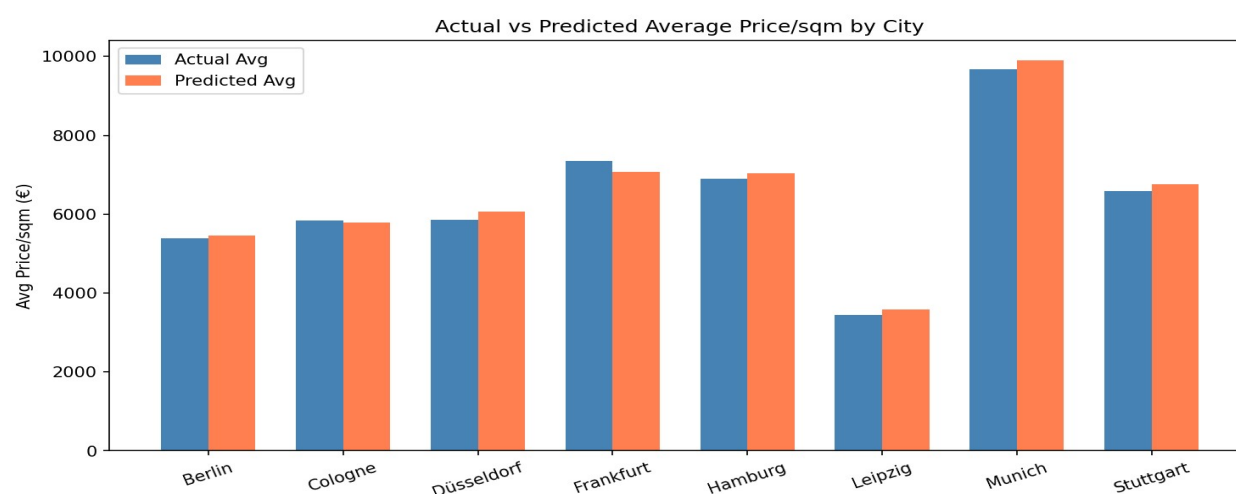


Figure 6: Actual vs predicted average price per sqm by city.

The model performs best in Hamburg ($R^2=0.79$) and Munich ($R^2=0.71$), cities with relatively homogeneous high-demand submarkets. Performance is lower in Düsseldorf ($R^2=0.38$), which may reflect greater heterogeneity in district quality within the city. Across all cities, mean predicted prices closely track actual means, confirming that the model is unbiased at the city level. This pattern suggests that future deployments of the model could benefit from city-specific calibration or hierarchical modelling approaches, rather than a single global model, particularly for less homogeneous markets such as Düsseldorf.

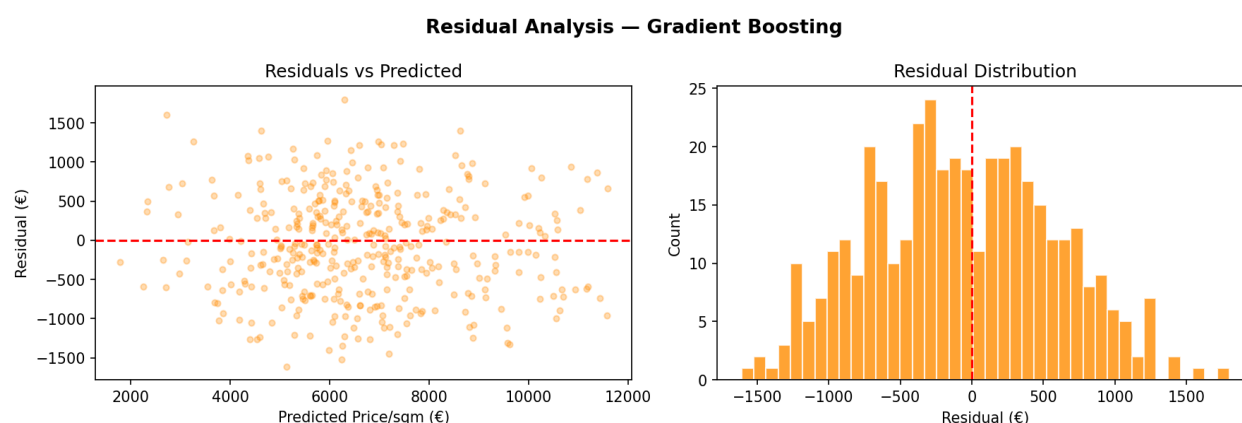


Figure 7: Residual analysis — residuals vs predicted (left) and residual distribution (right).

The residual analysis (Figure 7) shows that residuals are approximately normally distributed and centred near zero, with no systematic pattern against predicted values. This confirms that the model assumptions are broadly satisfied and that no major structural bias exists.

6. Conclusion and Future Work

This project successfully developed and evaluated a machine learning pipeline for predicting housing and rental prices in eight German metropolitan areas. The Gradient Boosting Regressor achieved the best performance ($R^2=0.89$, $MAE=€541/sqm$), substantially outperforming linear baselines. The analysis confirmed that location attributes — particularly the city itself, followed by distance to the city centre and property age — are the dominant price drivers, while property-level features play a secondary role.

These findings have practical implications for property valuation, investment analysis, and rental market regulation. A model of this kind could be deployed as an API-backed pricing tool for real estate platforms, enabling automated valuation at scale.

A key limitation of the current work is its reliance on a synthetically generated dataset rather than real transactional listings. While the generation process was carefully calibrated against published market statistics (see Section 4.1), the model's absolute predictions should be interpreted as illustrative of the achievable methodology rather than as production-ready price estimates. Nonetheless, the relative patterns identified — the dominance of location-based features over property-level attributes, and the clear advantage of non-linear ensemble methods over linear baselines — are consistent with findings widely reported in real estate analytics literature, and would be expected to persist once the pipeline is retrained on real listings data.

Future Work

- Integrate real-world data from Immobilienscout24 or ImmobilienAtlas APIs to replace the synthetic dataset with live market listings.
- Incorporate temporal features (listing date, year-on-year price trends) to enable time-series forecasting of future prices.
- Experiment with XGBoost and LightGBM architectures, which may further improve prediction accuracy.
- Develop a web application front-end allowing users to input property attributes and receive instant price estimates.
- Extend the geographic scope to include secondary German cities and rural areas to capture broader market dynamics.

Quantify prediction uncertainty (e.g., via quantile regression or conformal prediction) to provide confidence intervals alongside point estimates.
Conduct systematic hyperparameter tuning (e.g., grid search or Bayesian optimisation) to further improve on the current hand-tuned Gradient Boosting configuration.

7. References

Burkov, A. (2019) *The Hundred-Page Machine Learning Book*. Quebec City: Andriy Burkov.

IW Köln (2023) *Wohnungsmarktbericht Deutschland 2023*. [Online]. Available at: <https://www.iwkoeln.de> (Accessed: 23 June 2026).

McMillan, K. and Weyers, J. (2012) *How to Write Dissertations and Project Reports*. Harlow: Pearson Education.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12(1), pp. 2825-2830.

Statista (2023) *Average Housing Prices in Germany by City 2023*. [Online]. Available at: <https://www.statista.com> (Accessed: 23 June 2026).

VanderPlas, J. (2016) *Python Data Science Handbook: Essential Tools for Working with Data*. Sebastopol: O'Reilly Media.